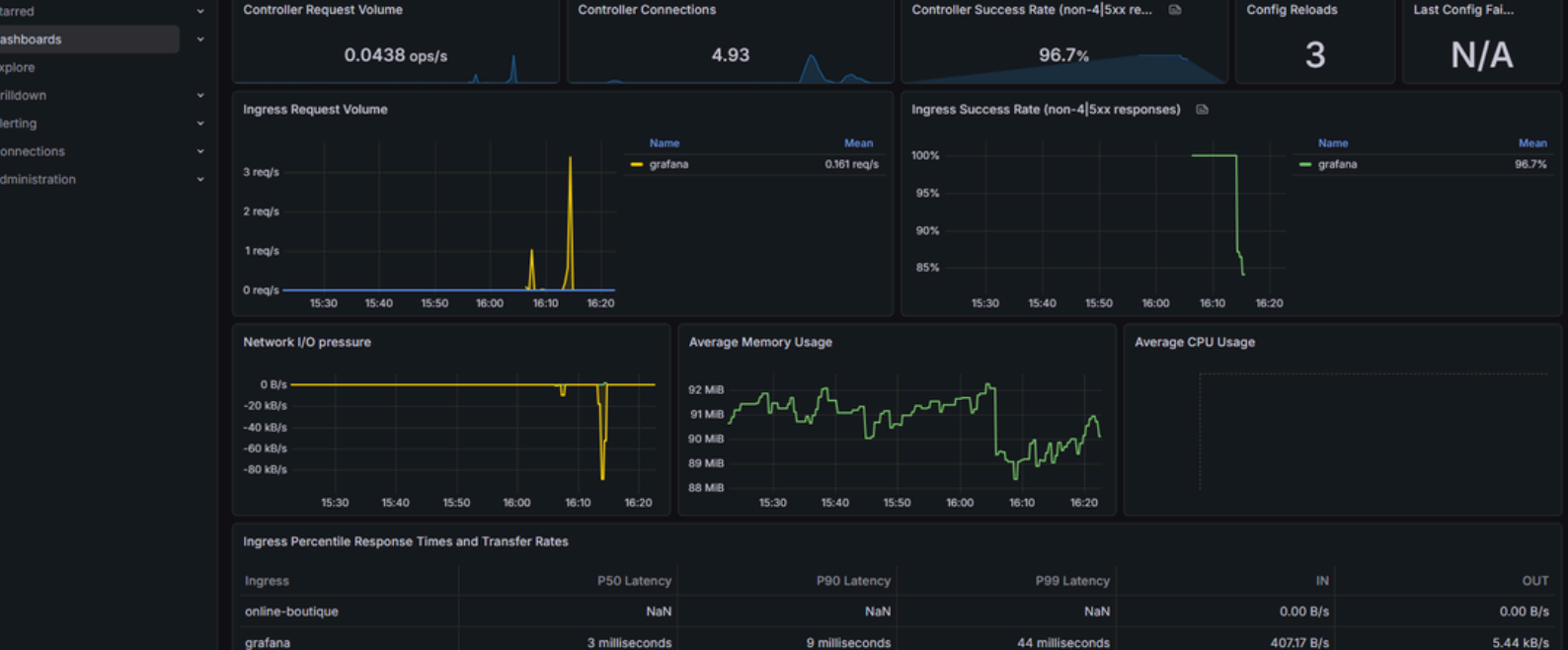




CASE STUDY

How I built a production observability platform on AWS EKS
— 96.7% SLO, \$6.50 total spend

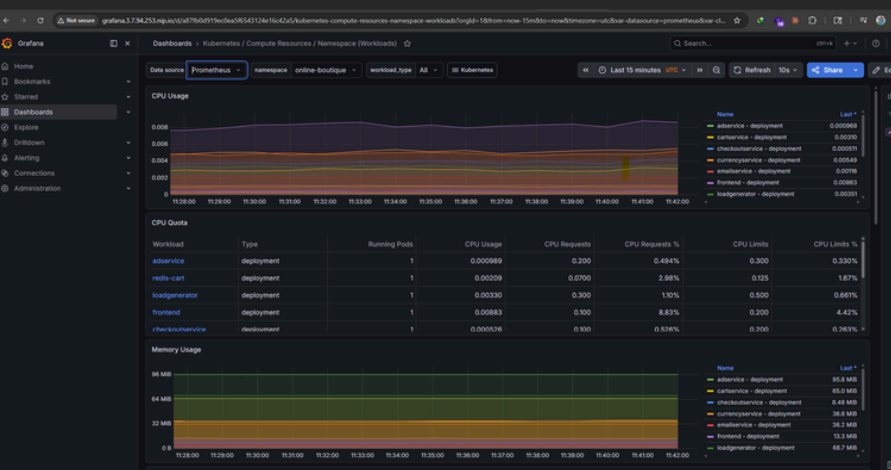
TABLE OF CONTENTS

TABLE OF CONTENTS	2
OVERVIEW	3
ARCHITECTURE & STACK	4
THE CHALLENGE	5
THE THREE SLOS	6
IMPLEMENTATION	7
LESSONS LEARNED	8
RESULTS	9
PERFORMANCE & COST	10
WHAT THIS PROVED	11
AVAILABLE FOR HIRE	12

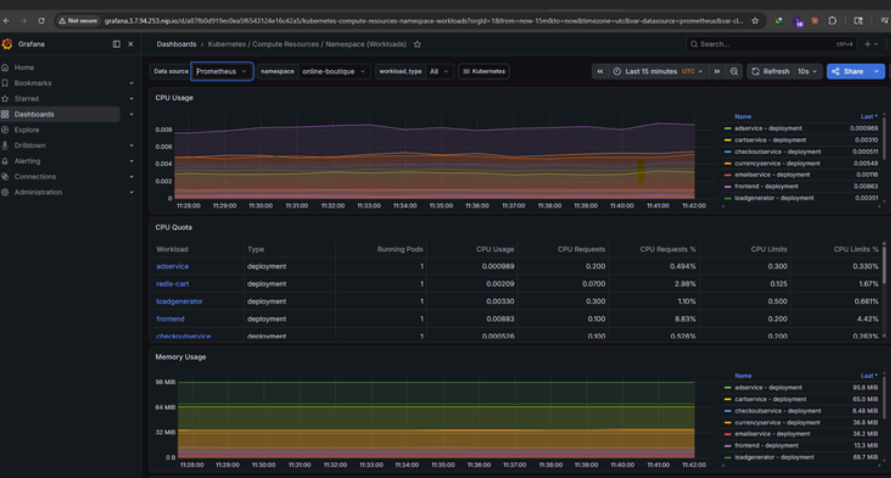
OVERVIEW

PulseHealth is a fictional 35-person B2B SaaS providing patient engagement and analytics dashboards to mid-sized clinics across North America.

BACKGROUND

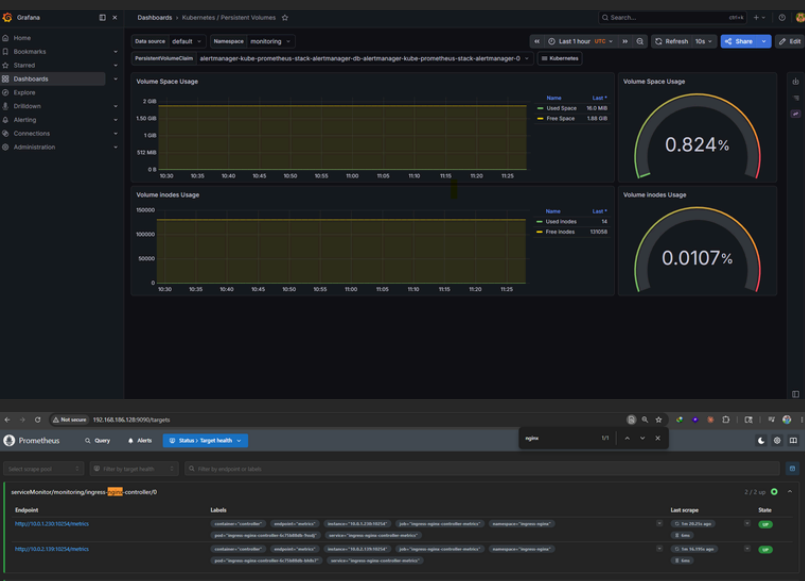


By mid-2025, PulseHealth's engineering team of 14 had outgrown the operations practices that worked when they were 10 people. Their backend microservices ran on Kubernetes, but their observability practice didn't keep pace with their growth. Three concrete pains had surfaced over the prior quarter.



The on-call rotation was experiencing severe alert fatigue — pages fired constantly, most non-actionable, and real incidents were being dismissed as noise. A growing percentage of outages were being reported by customers via support tickets before engineering's own monitoring detected them. And ahead of their Series B fundraising round, the CTO had no measurable answer to the board's question: "Is our service actually meeting customer expectations?"

ARCHITECTURE APPROACH



PulseHealth's platform was designed for AWS EKS in ap-south-1, with public ingress via a multi-AZ Network Load Balancer. The observability stack and the demo workload share the same ingress controller; Grafana is exposed via host-based routing using a wildcard subdomain.

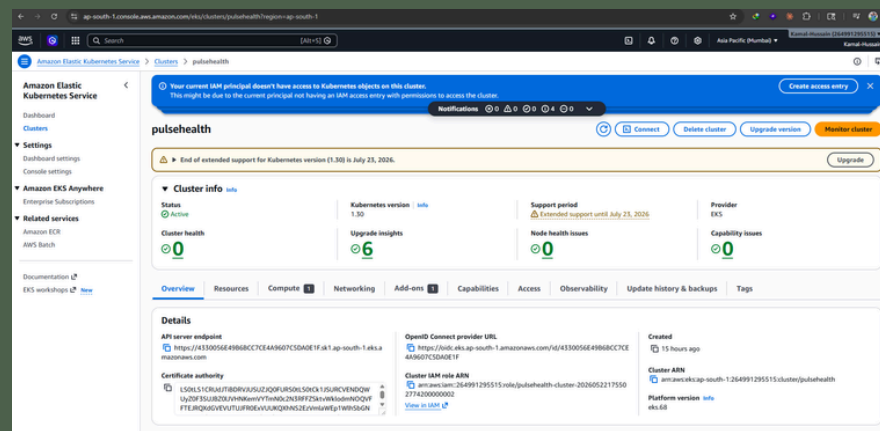
Infrastructure is fully Terraform-provisioned using community modules — no console clicks, no manual configuration. Helm with environment-specific value overlays gives clean separation between local development (kind) and production (EKS) without forking charts. EBS gp3 volumes (encrypted, CSI-provisioned via IRSA) back persistent storage for Prometheus, Alertmanager, and Grafana.

THE STACK

Infrastructure layer uses terraform-aws-modules/vpc/aws and terraform-aws-modules/eks/aws for VPC, EKS cluster, managed node groups, and the IRSA role for the EBS CSI driver. Single terraform apply brings everything online in ~15 minutes.

Observability layer is kube-prometheus-stack deployed via Helm — Prometheus, Alertmanager, Grafana, kube-state-metrics, and node-exporter in one chart with sane defaults. Ingress is ingress-nginx behind an AWS Network Load Balancer, provisioned automatically from Service annotations via the in-tree cloud-controller-manager.

Demo workload is Google's Online Boutique — 12 polyglot microservices (Go, Python, Node.js, C#, Java) communicating primarily over gRPC. Realistic enough to generate meaningful observability signal under load.



THE CHALLENGE

By mid-2025, PulseHealth's observability gap had crystallized into three concrete problems – each at a different level of the organization, each blocking real business outcomes.

Alert Fatigue

Pages fired constantly from static-threshold alerts and uptime checks — most non-actionable. The on-call rotation had begun dismissing alerts as noise. When a real 40-minute checkout outage hit, the triggering alert had fired hours earlier and been ignored. The cost: every real incident now arrived with an apology, and on-call engineers were burning out faster than the team could hire.

Customer- Discovered Outages

A growing percentage of incidents were being reported by customers — via support tickets and Twitter mentions — before engineering's own monitoring detected them. For a health-tech vendor where customer trust and uptime are core to the value proposition, this was an existential problem. Renewals were beginning to surface the concern in vendor reviews

No Measurable Answer for Leadership

Ahead of Series B fundraising, the CTO had no defensible answer to the board's question: "Is our service actually meeting customer expectations?" No SLO existed. No error budget. No language to discuss service health beyond raw uptime percentage. The lack of measurable service health was about to become a fundraising liability.

THE THREE SLO

INGRESS AVAILABILITY

99.0% over 30 days · Measured: 96.7%

P99 LATENCY

Target: <250ms · Measured: 44ms

ERROR BUDGET REMAINING

30-day window · alerts below 25%

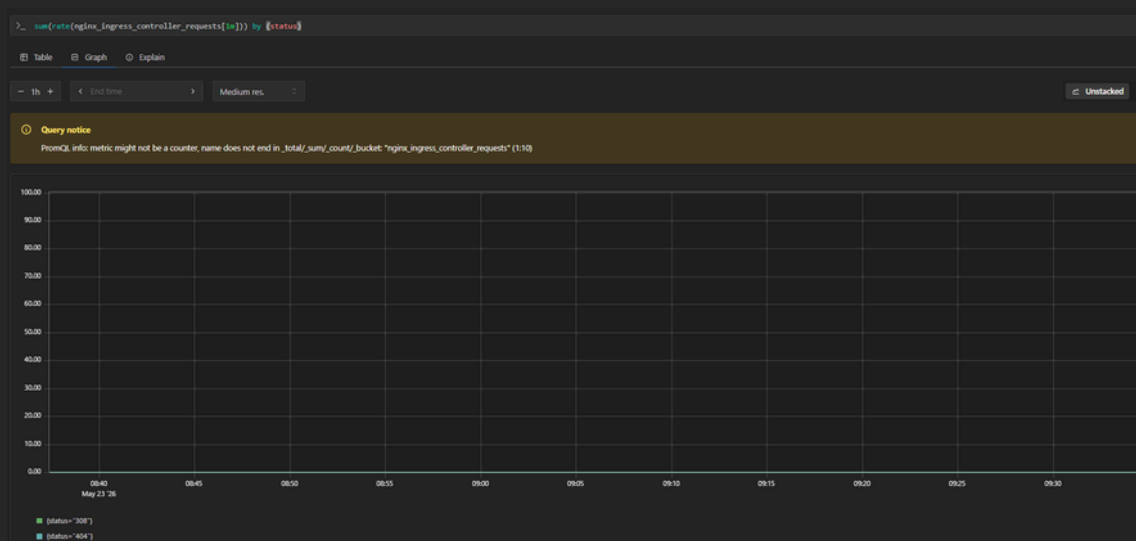
Three SLOs, each defined with measurable PromQL and tied to burn-rate alerts. Targets come from product expectations, not engineering preferences.

Each SLO has two alert tiers: a fast-burn rule for rapid outages, and a slow-burn rule for steady degradations — both from Google's SRE Workbook patterns.

Static thresholds either fire constantly or miss real degradations. Multi-window burn-rate alerts solve both — cutting noise while improving signal quality.

Each SLO is measured by a PromQL recording rule, so dashboards and alerts query the pre-computed value. Lower Prometheus load, faster alerts, no metric drift.

The result: alerts that mean something. When the on-call gets paged, the page is real — and the runbook URL tells them exactly what to do next.



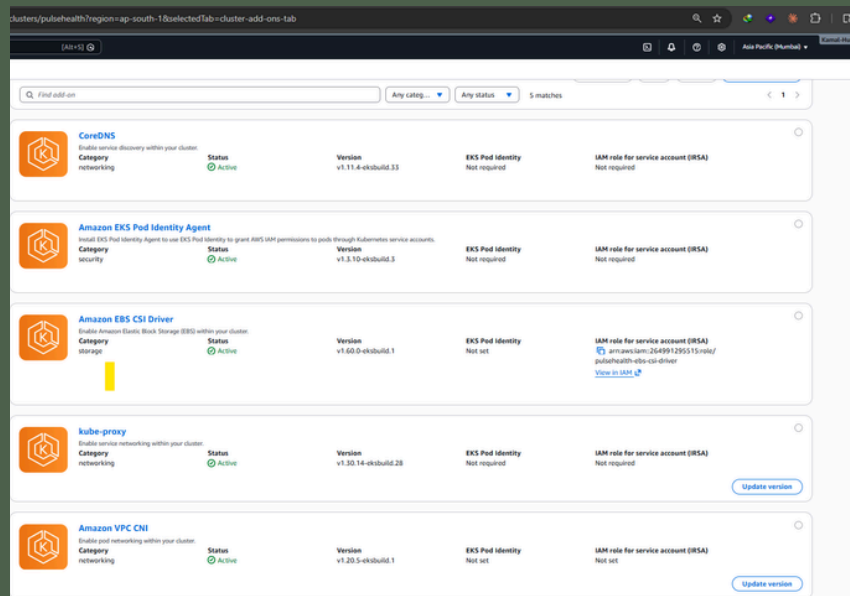
IMPLEMENTATION

The platform was built in two phases – local development on a kind cluster, then production deployment to AWS EKS. Same Helm chart, environment-specific value overlays, clean separation between dev and prod. Total end-to-end deploy time on EKS: roughly 30 minutes.

TWO-PHASE BUILD

Phase 1 — Local development on kind. Built the full observability stack on a single-node kind cluster: kube-prometheus-stack installation, ServiceMonitor and PrometheusRule CRDs, ingress-nginx integration, PromQL queries for the SLOs, and recording rules. Used as the development substrate — the EKS deployment reuses the same chart with overlay values, so dev/prod stay in sync without forking.

Phase 2 — Production on EKS. terraform apply provisions VPC, EKS, node group, OIDC provider, and the IRSA role for EBS CSI in ~15 minutes. Helm installs the observability stack with the EKS overlay, then ingress-nginx — AWS provisions the NLB automatically from Service annotations. Online Boutique manifests and the Grafana public-access Ingress applied last. End-to-end: ~30 minutes from terraform apply to a live dashboard URL.



LESSONS LEARNED

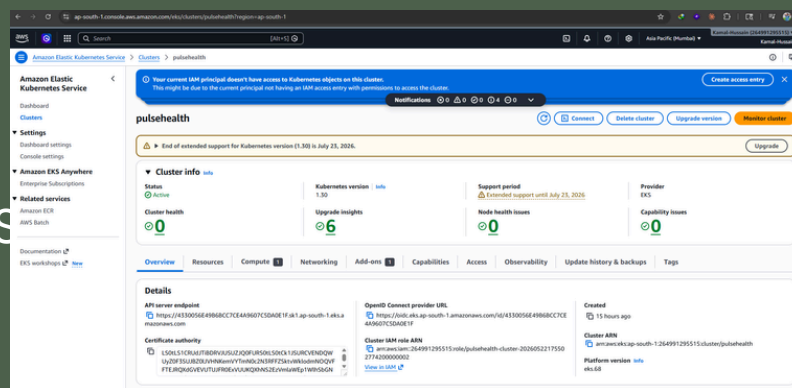
THREE REAL FAILURES ENCOUNTERED DURING BUILD AND TEARDOWN. DOCUMENTED SO THE NEXT ENGINEER – INCLUDING FUTURE ME – DOESN'T REPEAT THEM.

TERRAFORM DESTROY HANGS ON K8S-OWNED RESOURCES

When a Kubernetes Service of type LoadBalancer is created, the cluster's cloud-controller-manager provisions an AWS load balancer and attaches ENIs to the VPC's subnets. These resources are owned by Kubernetes, not Terraform's state. If terraform destroy kills the cluster before the LoadBalancer Service is explicitly deleted, the cleanup never runs – and Terraform fails because subnets still hold those ENIs.

Fix: delete LoadBalancer Services and PVCs first, wait 3 minutes, then terraform destroy.

Documented as a full procedure in DESTROY-RUNBOOK.md.



LESSON 3 – AWS HAS TWO ELB APIS

After multiple terraform destroy failures, aws elbv2 describe-load-balancers returned empty – but the VPC still had ENIs labeled ELB... blocking subnet deletion. The elbv2 API only lists ALBs and NLBs. Classic Load Balancers live under elb (v1). The Online Boutique manifests included a frontend-external Service of type LoadBalancer without annotations – silently spawning a CLB that sat consuming ENIs.

Fix: always check both ELB APIs plus enumerate ENIs directly before destroying a VPC. Either-or queries leave Classic LBs hidden.



Grafana crashed repeatedly with restart counts climbing over hours. kubectl describe pod revealed Last State: Terminated, Reason: OOMKilled, Exit Code: 137. The Helm value of 256Mi memory was too aggressive – under real dashboard query load, memory consistently spiked past it. Kernel sent SIGKILL, kubelet restarted, loop continued.

Fix: Helm upgrade with --reuse-values bumping memory to 768Mi. Stable immediately. Lesson: resource limits from "looks reasonable" guesses are a leading cause of production instability.

RESULTS

Under sustained load against Google's Online Boutique (12 polyglot microservices, gRPC-heavy traffic), the platform delivered measurable, defensible service health – at a deliberately disciplined cost.

INGRESS
SUCCESS
RATE

96.7%

P99 LATENCY

44MS

TOTAL AWS
SPEND

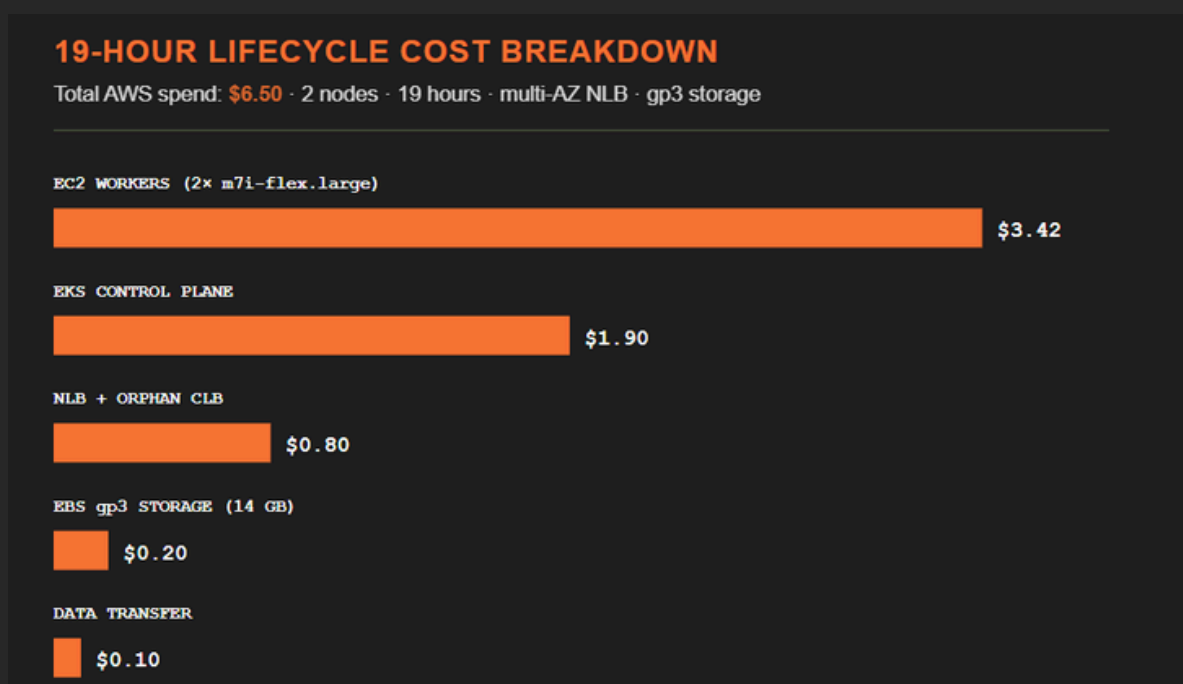
\$6.50

Measured against real workload — public traffic through AWS NLB → ingress-nginx → application pods, with latency percentiles from `nginx_ingress_controller_request_duration_seconds`. No synthetic load, no benchmark fudging.

Cost reflects 19 hours of EKS control plane, 2× m7i-flex.large workers, multi-AZ NLB, gp3 storage, and data transfer. Teardown was treated as part of the engagement — the destroy procedure documented in `DESTROY-RUNBOOK.md`.

PERFORMANCE & COST

The platform was built and torn down across roughly 19 hours of cluster runtime. Latency held steady through sustained Online Boutique traffic; AWS spend stayed disciplined throughout.



Total spend for the full lifecycle — provision, deploy, validate under load, document, tear down cleanly — was roughly \$6.50. The teardown procedure handles orphan Kubernetes-owned cloud resources (LoadBalancer ENIs, EBS-backed PVCs) explicitly, because terraform destroy alone leaves these stranded. Both create and destroy are treated as production procedures and documented as such. The result: a real platform demonstration, with discipline equivalent to a paid client engagement, at a cost that's a rounding error on most AWS bills.

WHAT THIS PROVED

This was a self-funded engagement built to consultancy standards – fictional client, real engineering. The artifacts speak for themselves: a working platform, measured outcomes under load, documented failures, clean teardown. The principles below shaped how it was built.

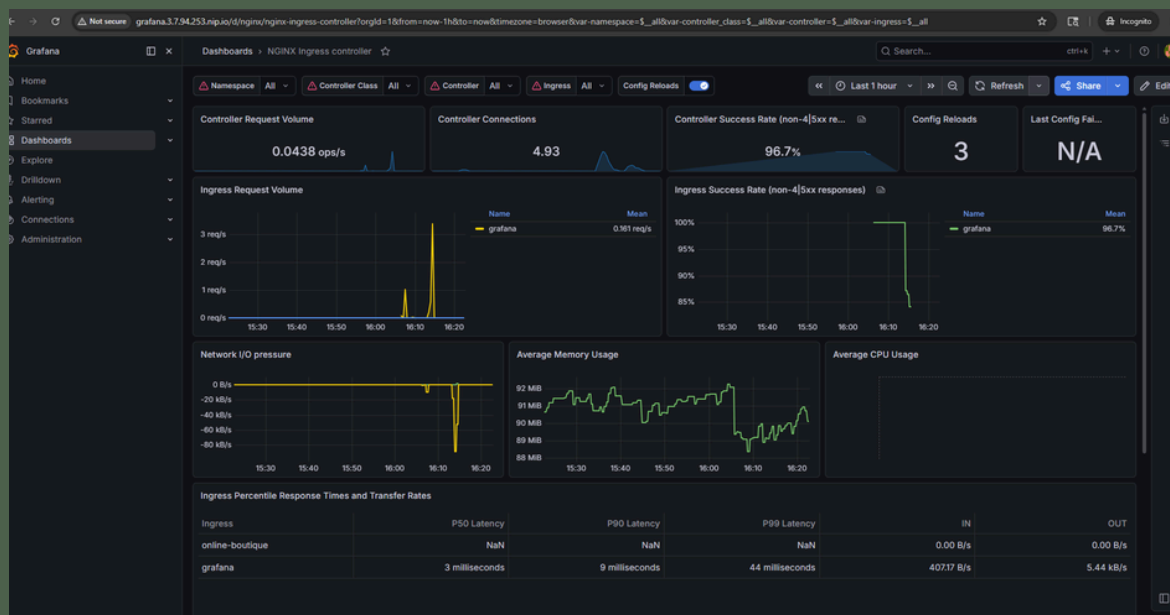
ENGINEERING PRINCIPLES

"Build the destroy procedure as carefully as you build the create procedure. Both go in version control. Both get tested. terraform apply and terraform destroy aren't symmetric — apply assumes a clean slate, destroy assumes a cooperative environment. The latter is harder, and worth equal engineering investment."

— FROM LESSONS.MD · PULSEHEALTH PROJECT META-LESSON

AVAILABLE FOR HIRE

Need similar work — observability, AWS EKS platforms, Kubernetes operations, or Terraform infrastructure? Let's talk about what's broken and how I'd fix it.



CONTACT

[GITHUB: KAMAL-HUSSAIN](#)

[UPWORK — AVAILABLE FOR HIRE](#)